

Raport

June 29, 2025

Jędrzej Bojanowski, Kacper Borys, Nadia Gałkiewicz, Igor Kociszewski, Michał Wrona

```
[1]: import pandas as pd
import numpy as np
import mysql.connector as sql
import seaborn as sns
import matplotlib.pyplot as plt
import scipy.stats as stats
from IPython.display import display, Markdown
from collections import defaultdict
from sqlalchemy import create_engine, text
import warnings
warnings.filterwarnings('ignore')
```

```
[2]: sns.set_theme(style="whitegrid")
```

```
[3]: db = sql.connect(
    host="giniewicz.it",
    user="team07",
    password="te@mlo7",
    database="team07",
    port=3306,
)

db.is_connected()
```

```
[3]: True
```

```
[4]: cursor = db.cursor()
```

1 Analiza danych

1.1 Znajdź najpopularniejsze rodzaje wypraw, porównaj koszty i zyski, czy są opłacalne?

Wyberzmy 5 najpopularniejszych wypraw.

```
[5]: top_5_trips_query = """
SELECT name, COUNT(*) as trip_count
FROM Trips
GROUP BY name
ORDER BY COUNT(*) DESC
LIMIT 5
"""

cursor.execute(top_5_trips_query)
trips = cursor.fetchall()
top_5_trips_df = pd.DataFrame(trips, columns=[i[0] for i in cursor.description])
```

```
[6]: top_5_trips_kind_customers_df = pd.DataFrame()
top_5_trips_kind_details_df = pd.DataFrame()

for name in top_5_trips_df['name'].values:
    named_trips_query = f"""
SELECT trip_id, organization_cost, launch_date, name
FROM Trips
WHERE name = '{name}'
"""

    cursor.execute(named_trips_query)
    trip_data = cursor.fetchall()

    top_5_trips_kind_details_df = pd.concat([top_5_trips_kind_details_df, pd.
↳ DataFrame(trip_data, columns=[i[0] for i in cursor.description])])

    for trip_id, *_ in trip_data:

        trip_query = f"""
SELECT participant_id, pesel, trip_id, ticket_price
FROM TripParticipants
WHERE trip_id = {trip_id}
"""

        cursor.execute(trip_query)
        participants = cursor.fetchall()
        top_5_trips_kind_customers_df = pd.
↳ concat([top_5_trips_kind_customers_df, pd.DataFrame(participants,
↳ columns=[i[0] for i in cursor.description])])
```

```
[7]: zyski_koszta = top_5_trips_kind_customers_df.join(
    top_5_trips_kind_details_df.set_index('trip_id'),
    on='trip_id',
    lsuffix='_customer',
    rsuffix='_details'
).groupby('trip_id').agg(
    _payout = ('ticket_price', 'sum'),
    _cost = ('organization_cost', 'first'),
```

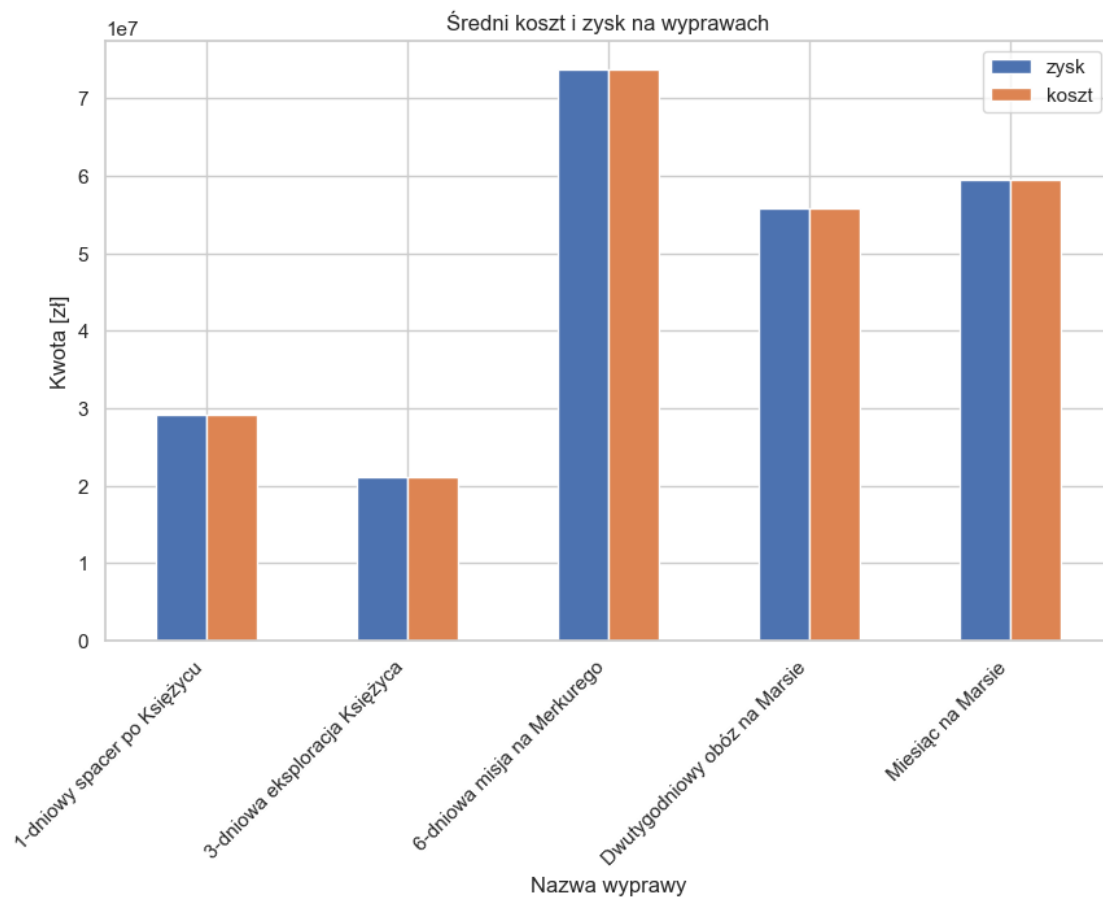
```

    name = ('name', 'first'),
).groupby('name').agg(
    zysk = ('_payout', 'mean'),
    koszt = ('_cost', 'mean'),
)

ax = zyski_koszta.plot.bar(
    y=['zysk', 'koszt'],
    title='Średni koszt i zysk na wyprawach',
    ylabel='Kwota [zł]',
    xlabel='Nazwa wyprawy',
    figsize=(10, 6),
)

ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right');

```

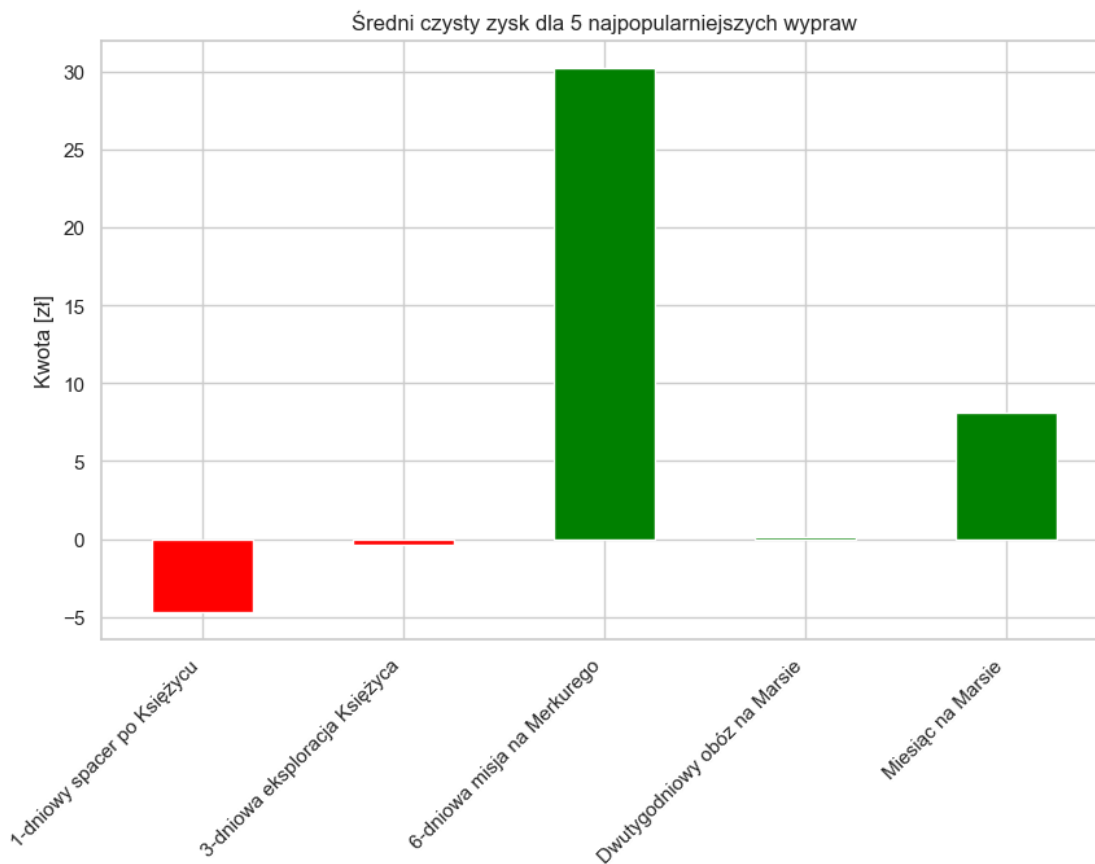


Widzimy, że koszt i zysk wypraw jest praktycznie ten sam.

```
[8]: zyski_koszta['czysty_zysk'] = zyski_koszta['zysk'] - zyski_koszta['koszt']
      colors = ['red' if x < 0 else 'green' for x in zyski_koszta['czysty_zysk']]
      ax = zyski_koszta['czysty_zysk'].plot.bar(
          title='Średni czysty zysk dla 5 najpopularniejszych wypraw',
          ylabel='Kwota [zł]',

          figsize=(10, 6),
          color=colors
      )
      ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right');
      ax.set_xlabel(None)
```

```
[8]: Text(0.5, 0, '')
```



Czysty zysk jest naprawdę marginalny w porównaniu z kosztami wyprawy dla 5 najpopularniejszych wypraw. Jak wygląda to w kontekście wszystkich wypraw?

```
[9]: payout_from_all_query = """
      SELECT
          SUM(ticket_price) AS total_payout, Trips.trip_id, name, organization_cost
```

```

FROM
    TripParticipants
JOIN Trips ON TripParticipants.trip_id = Trips.trip_id
GROUP BY
    Trips.trip_id
"""

cursor.execute(payout_from_all_query)
payout_from_all = cursor.fetchall()
payout_from_all_df = pd.DataFrame(payout_from_all, columns=[i[0] for i in cursor.description])

```

```

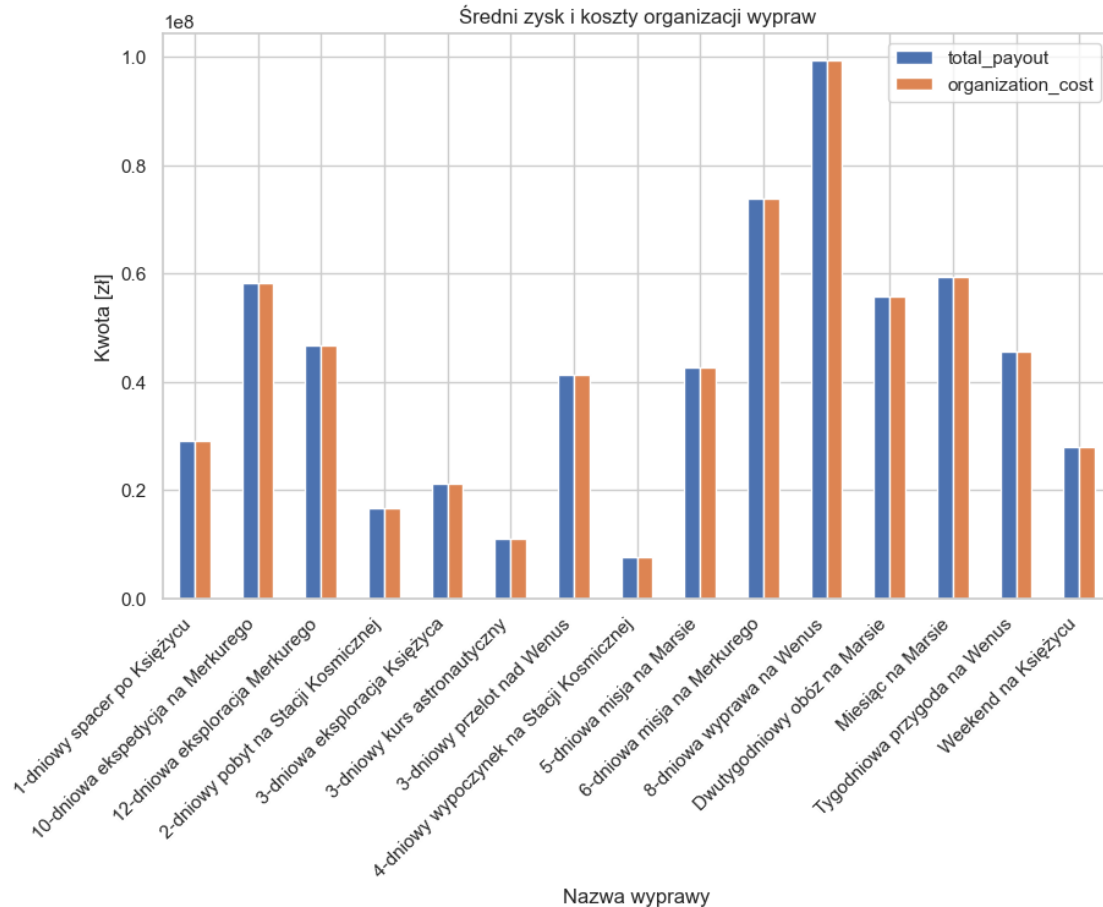
[10]: payout_from_all_df['czysty_zysk'] = payout_from_all_df['total_payout'] -
      payout_from_all_df['organization_cost'].astype(float)

```

```

[11]: payout_from_all_df.groupby('name').agg(
      czysty_zysk=('czysty_zysk', 'mean'),
      total_payout=('total_payout', 'mean'),
      organization_cost=('organization_cost', 'mean'),
    ).plot.bar(
      y=['total_payout', 'organization_cost'],
      title='Średni zysk i koszty organizacji wypraw',
      ylabel='Kwota [zł]',
      xlabel='Nazwa wyprawy',
      figsize=(10, 6),
    )
plt.xticks(rotation=45, ha='right');

```



Brak znaczącego zysku nie ogranicza się tylko do wcześniej wymienionych wypraw ale do **wszystkich**. Wygląda na to, że firma nie ma planów na jakiekolwiek inwestycje i w związku z tym nie podności marży.

1.2 Sporządź wykres liczby obsługiwanych klientów w każdym miesiącu działalności firmy, czy firma rośnie, czy podupada?

```
[25]: participants_query = """
SELECT
    COUNT(*) AS participants_count,
    launch_date
FROM
    TripParticipants
JOIN Trips ON TripParticipants.trip_id = Trips.trip_id
GROUP BY
    launch_date
ORDER BY
    launch_date
"""
```

```

cursor.execute(participants_query)
participants = cursor.fetchall()
participants_df = pd.DataFrame(participants, columns=[i[0] for i in cursor.
    ↳description])
participants_df['launch_date'] = pd.to_datetime(participants_df['launch_date'])
participants_df.set_index('launch_date', inplace=True)

plt.figure(figsize=(12, 6))

sns.scatterplot(
    data=participants_df,
    x=participants_df.index,
    y='participants_count',
    color='blue',
    alpha=0.7
)
plt.xticks(rotation=45, ha='right');

thirty_days_mean = participants_df['participants_count'].rolling(window=30).
    ↳mean().dropna()

sns.lineplot(
    data=thirty_days_mean,
    color='orange',
    label='Średnia liczba uczestników w ciągu 30 dni'
)

plt.title('Wyprawy uczestników w okresie działania firmy')
plt.xticks(rotation=45, ha='right');

print(thirty_days_mean)

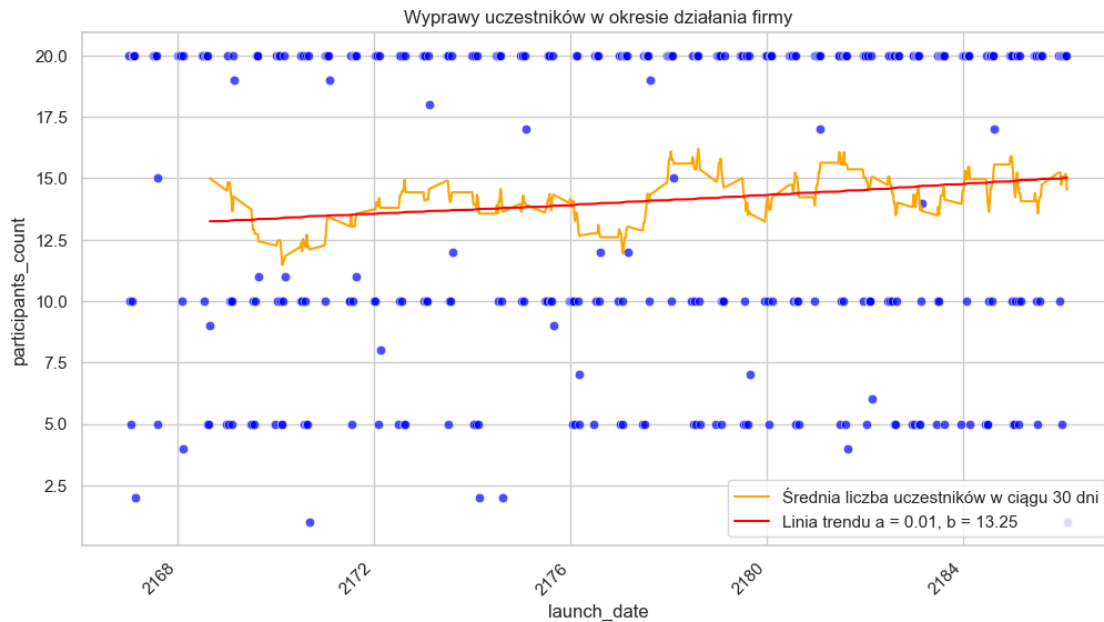
a, b = stats.linregress(
    np.arange(len(thirty_days_mean)),
    thirty_days_mean
)[:2]

trendline = a * np.arange(len(thirty_days_mean)) + b
plt.plot(
    thirty_days_mean.index,
    trendline,
    color='red',
    label=f'Linia trendu a = {a:.2f}, b = {b:.2f}'
)

```

```
plt.legend()
plt.show()
```

```
launch_date
2168-08-25    15.000000
2168-12-29    14.500000
2169-01-05    14.833333
2169-01-12    14.833333
2169-01-19    14.833333
...
2186-01-12    15.066667
2186-01-19    15.066667
2186-01-26    15.166667
2186-02-02    15.166667
2186-02-09    14.533333
Name: participants_count, Length: 340, dtype: float64
```



Liczba uczestników wypraw waha się często pomiędzy 20, 10 i 5. Nie ma widocznych anomalii, dane są bardzo regularne. Współczynnik kierunkowy uzyskany przez regresję liniową $a = 0.01$ wskazują na mały wzrost zainteresowania firmą w ciągu lat. Czyli firma nie traci klientów, a nawet ich zyskuje. W takim wypadku pozostaje tylko zbadać które wyprawy ponownie przyciągają klientów.

1.3 Sprawdź, po których wycieczkach klienci wracają na kolejne, a po których mają dość i więcej ich nie widzicie. Czy są takie, które być może powinny zniknąć z oferty?

```
[ ]: query_returned = """
SELECT pesel, trip_id, name AS trip_name
FROM TripParticipants
JOIN Trips USING(trip_id)
"""

df = pd.read_sql(query_returned, db)

client_trips = df.groupby('pesel').agg(list)

trip_stats = defaultdict(lambda: {"returned": 0, "stopped": 0})
for trips in client_trips['trip_id']:
    for i, trip_id in enumerate(trips):
        if i < len(trips) - 1:
            trip_stats[trip_id]['returned'] += 1
        else:
            trip_stats[trip_id]['stopped'] += 1

trip_names = df.drop_duplicates(subset='trip_id')[['trip_id', 'trip_name']].
    ↪set_index('trip_id')

result_df = pd.DataFrame.from_dict(trip_stats, orient='index')
result_df.index.name = 'trip_id'
result_df = result_df.join(trip_names)

result_df = result_df[['trip_name', 'returned', 'stopped']].
    ↪sort_values(by='returned', ascending=False)

returned, stopped = sum(result_df["returned"]), sum(result_df["stopped"])
print(f"Procent osób które wróciły na kolejną wycieczkę: {returned/
    ↪(stopped+returned)*100:.3f} %\n")
display(result_df)
```

Procent osób które wróciły na kolejną wycieczkę: 4.780 %

	trip_name	returned	stopped
trip_id			
45	12-dniowa eksploracja Merkurego	6	14
17	3-dniowy przelot nad Wenus	5	15
4	1-dniowy spacer po Księżycu	4	6
25	3-dniowy przelot nad Wenus	4	16
5	Miesiąc na Marsie	4	16
...	...	...	...

307	3-dniowa eksploracja Księżyca	0	10
341	1-dniowy spacer po Księżycu	0	10
136	1-dniowy spacer po Księżycu	0	10
218	3-dniowa eksploracja Księżyca	0	10
369	8-dniowa wyprawa na Wenus	0	1

[369 rows x 3 columns]

Ze każdej wycieczki, średnio jedynie 4.780 % uczestników decyduje się, żeby wybrać się na kolejną wyprawę. Zobaczmy jak ten wynik będzie zależeć od celu podróży, czasu pobytu oraz statku kosmicznego, którym lecą.

```
[43]: query_by_destination = """
SELECT tp.pesel, tp.trip_id, t.name AS trip_name, t.destination_id
FROM TripParticipants tp
JOIN Trips t ON tp.trip_id = t.trip_id
"""

df = pd.read_sql(query_by_destination, db)

df_destinations = pd.read_sql("SELECT destination_id, name FROM Destinations", db)
df_destinations = df_destinations.set_index('destination_id')

client_trips = df.groupby('pesel').agg(list)
destination_stats = defaultdict(lambda: {"returned": 0, "stopped": 0})
trip_to_destination = df.set_index('trip_id')['destination_id'].to_dict()

for trips in client_trips['trip_id']:
    for i, trip_id in enumerate(trips):
        destination = trip_to_destination[trip_id]
        if i < len(trips) - 1:
            destination_stats[destination]['returned'] += 1
        else:
            destination_stats[destination]['stopped'] += 1

result_df = pd.DataFrame.from_dict(destination_stats, orient='index')
result_df.index.name = 'Destination'
result_df = result_df.sort_values(by='returned', ascending=False)

ax = result_df.plot(kind='bar', stacked=True, figsize=(10, 6), color=["green", "red"])
xticks = range(len(result_df))
labels = df_destinations.loc[result_df.index, 'name']
plt.xticks(ticks=xticks, labels=labels, rotation=45)
plt.title("Ilość osób którzy wrócili, w zależności od celu podróży")
plt.xlabel("Cel podróży")
plt.legend(["wrócili", "nie wrócili"])
```

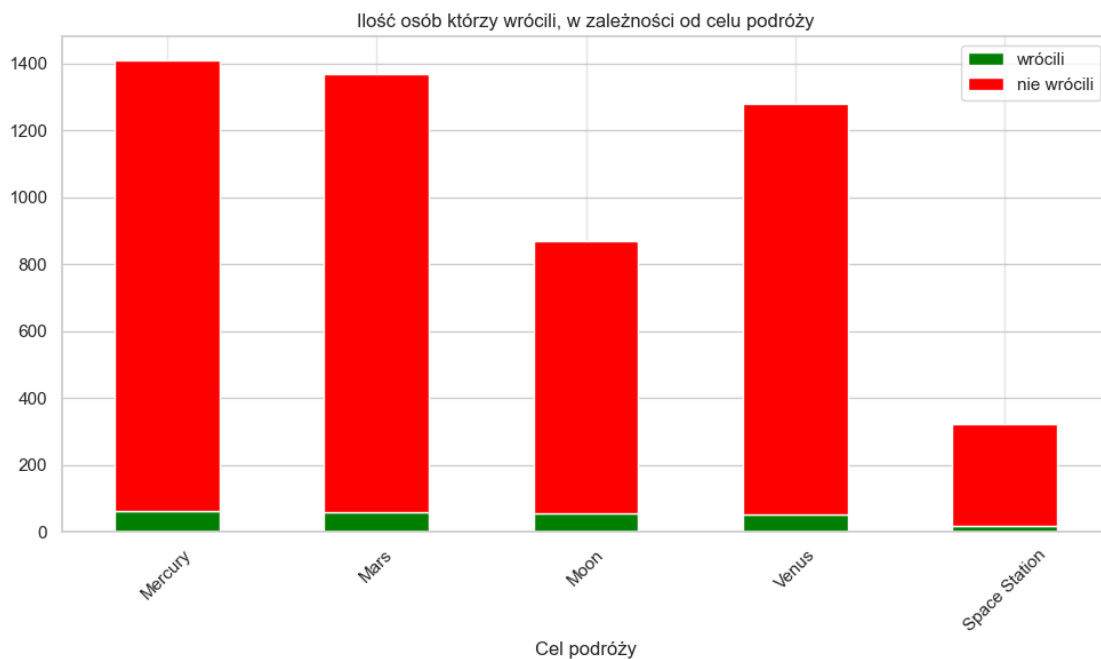
```

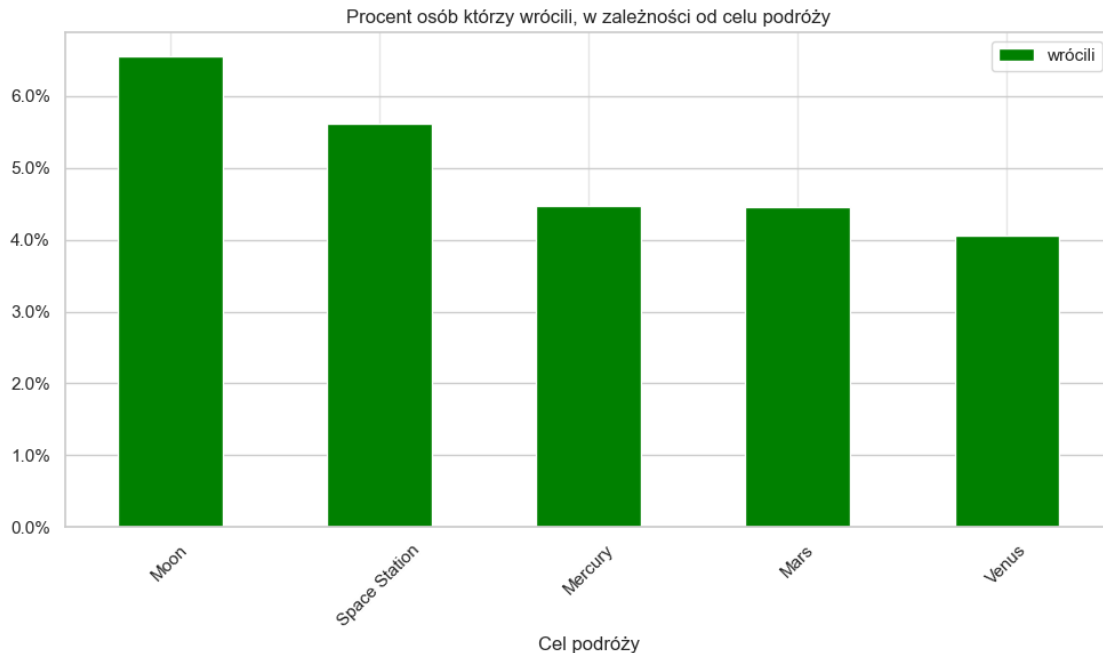
plt.tight_layout()
plt.grid(axis='x', alpha=0.5)
plt.show()

df_stats = pd.DataFrame.from_dict(destination_stats, orient='index')
df_stats.index.name = 'Destination'
df_stats['total'] = df_stats['returned'] + df_stats['stopped']
df_stats['returned_pct'] = df_stats['returned'] / df_stats['total']
df_stats['stopped_pct'] = df_stats['stopped'] / df_stats['total']
df_stats = df_stats.sort_values(by='returned_pct', ascending=False)

plot_df = df_stats[['returned_pct']]
ax = plot_df.plot(kind='bar', stacked=True, figsize=(10, 6), color=['green', 'red'])
xticks = range(len(plot_df))
labels = df_destinations.loc[plot_df.index, 'name']
plt.xticks(ticks=xticks, labels=labels, rotation=45)
ax.set_yticklabels(['{:, .1%}'.format(x) for x in ax.get_yticks()])
plt.title("Procent osób którzy wrócili, w zależności od celu podróży")
plt.xlabel("Cel podróży")
plt.legend(["wrócili", "nie wrócili"])
plt.tight_layout()
plt.grid(axis='x', alpha=0.5)
plt.show()

```





Celami podróży, które najbardziej zachęciły do ponownego wyruszenia na wyprawę są Księżyc na poziomie 6.6 % i stacja kosmiczna na poziomie 5.6 %. Dodatkowo są to najmniej popularne kierunki, więc można spróbować podnieść marketing do tych destynacji.

```
[ ]: query_by_duration = """
SELECT tp.pesel, tp.trip_id, t.name AS trip_name, t.duration_of_stay
FROM TripParticipants tp
JOIN Trips t ON tp.trip_id = t.trip_id
ORDER BY tp.pesel, tp.trip_id
"""

df = pd.read_sql(query_by_duration, db)

client_trips = df.groupby('pesel').agg(list)
duration_stats = defaultdict(lambda: {"returned": 0, "stopped": 0})
trip_duration = df.set_index('trip_id')['duration_of_stay'].to_dict()

for trips in client_trips['trip_id']:
    for i, trip_id in enumerate(trips):
        duration = trip_duration[trip_id]
        if i < len(trips) - 1:
            duration_stats[duration]['returned'] += 1
        else:
            duration_stats[duration]['stopped'] += 1

result_df = pd.DataFrame.from_dict(duration_stats, orient='index')
```

```

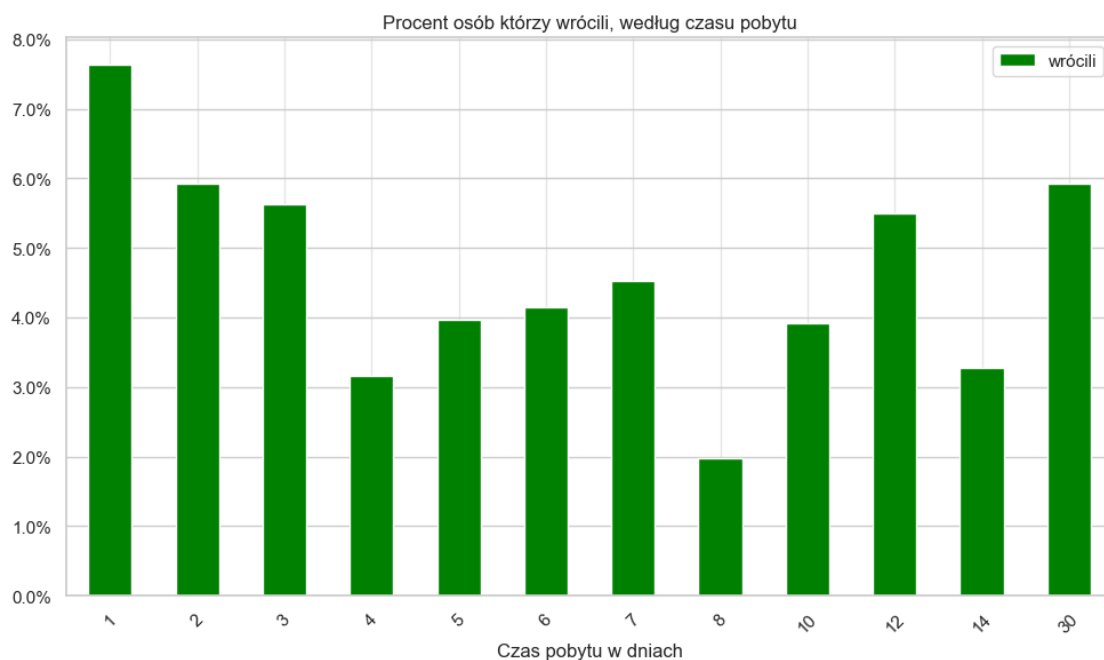
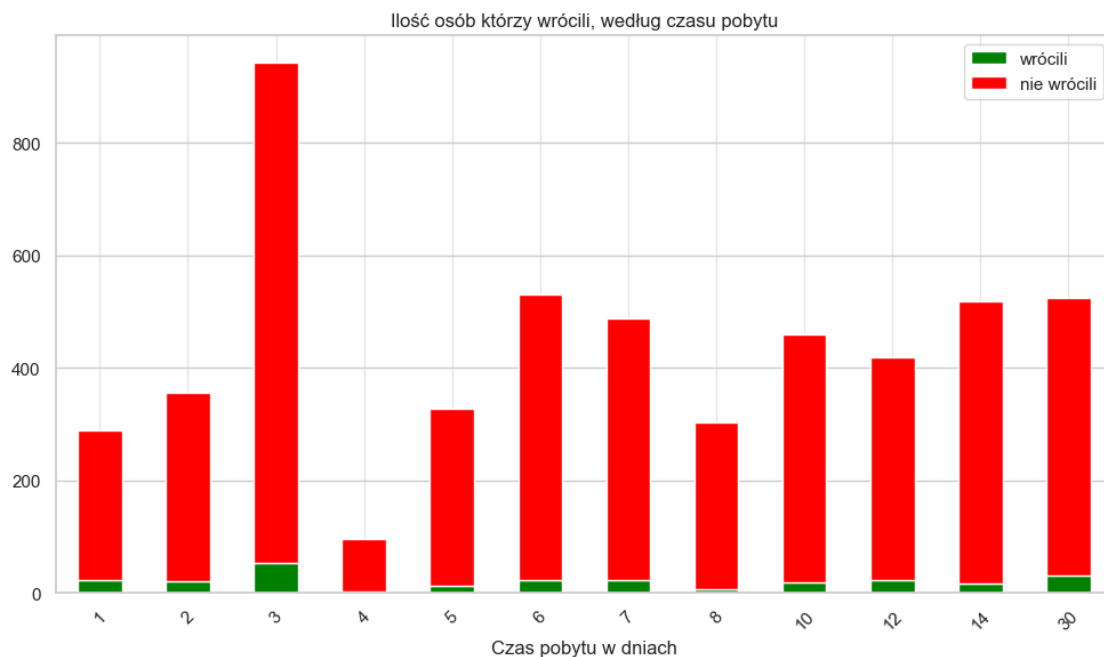
result_df.index.name = 'Duration'
result_df = result_df.sort_values(by='Duration', ascending=True)

ax = result_df.plot(kind='bar', stacked=True, figsize=(10, 6), color=["green", "red"])
plt.title("Ilość osób którzy wrócili, w zależności od czasu pobytu")
plt.xlabel("Czas pobytu w dniach")
plt.xticks(rotation=45)
plt.legend(["wrócili", "nie wrócili"])
plt.tight_layout()
plt.grid(axis='x', alpha=0.5)
plt.show()

df_stats = pd.DataFrame.from_dict(duration_stats, orient='index')
df_stats.index.name = 'Duration'
df_stats['total'] = df_stats['returned'] + df_stats['stopped']
df_stats['returned_pct'] = df_stats['returned'] / df_stats['total']
df_stats['stopped_pct'] = df_stats['stopped'] / df_stats['total']
df_stats = df_stats.sort_values(by='Duration', ascending=True)

plot_df = df_stats[['returned_pct']]
ax = plot_df.plot(kind='bar', stacked=True, figsize=(10, 6), color=['green', "red"])
plt.title("Procent osób którzy wrócili, w zależności od czasu pobytu")
plt.xticks(rotation=45)
ax.set_yticklabels(['{:, .1%}'.format(x) for x in ax.get_yticks()])
plt.xlabel("Czas pobytu w dniach")
plt.legend(["wrócili", "nie wrócili"])
plt.tight_layout()
plt.grid(axis='x', alpha=0.5)
plt.show()

```



Zarówno ilość ja i procent osób, którzy posownie wybierają się na wyprawę jest bardzo zróżnicowana. Nie widać konkretnego trendu. Najbardziej popularne są wycieczki 3 dniowa, natomiast najbardziej zachęcające do powrotu są wycieczki 1, 2, 3, 12 oraz 30-dniowe - powyżej 5 %, a najmniej 4, 8 i 14-dniowe - poniżej 3.5 %.

```

[49]: query_by_spaceship = """
SELECT tp.pesel, tp.trip_id, t.name AS trip_name, t.spaceship_id
FROM TripParticipants tp
JOIN Trips t ON tp.trip_id = t.trip_id
ORDER BY tp.pesel, tp.trip_id
"""

df = pd.read_sql(query_by_spaceship, db)

df_spaceships = pd.read_sql("SELECT spaceship_id, name FROM Spaceships", db)
df_spaceships = df_spaceships.set_index('spaceship_id')

client_trips = df.groupby('pesel').agg(list)
spaceship_stats = defaultdict(lambda: {"returned": 0, "stopped": 0})
trip_by_spaceship = df.set_index('trip_id')['spaceship_id'].to_dict()

for i in range(len(df_spaceships)):
    spaceship_stats[i+1]['returned'] = 0
    spaceship_stats[i+1]['stopped'] = 0

for trips in client_trips['trip_id']:
    for i, trip_id in enumerate(trips):
        spaceship = trip_by_spaceship[trip_id]
        if i < len(trips) - 1:
            spaceship_stats[spaceship]['returned'] += 1
        else:
            spaceship_stats[spaceship]['stopped'] += 1

result_df = pd.DataFrame.from_dict(spaceship_stats, orient='index')
result_df.index.name = 'Spaceship'
result_df = result_df.sort_values(by='returned', ascending=False)

ax = result_df.plot(kind='bar', stacked=True, figsize=(10, 6), color=["green", "red"])
xticks = range(len(result_df))
labels = df_spaceships.loc[result_df.index, 'name']
plt.xticks(ticks=xticks, labels=labels, rotation=45, ha='right')
plt.title("Ilość osób którzy wrócili, w zależności od statku kosmicznego")
plt.xlabel("Statek kosmiczny")
plt.legend(["wrócili", "nie wrócili"])
plt.tight_layout()
plt.grid(axis='x', alpha=0.5)
plt.show()

df_stats = pd.DataFrame.from_dict(spaceship_stats, orient='index')
df_stats.index.name = 'Spaceship'
df_stats['total'] = df_stats['returned'] + df_stats['stopped']

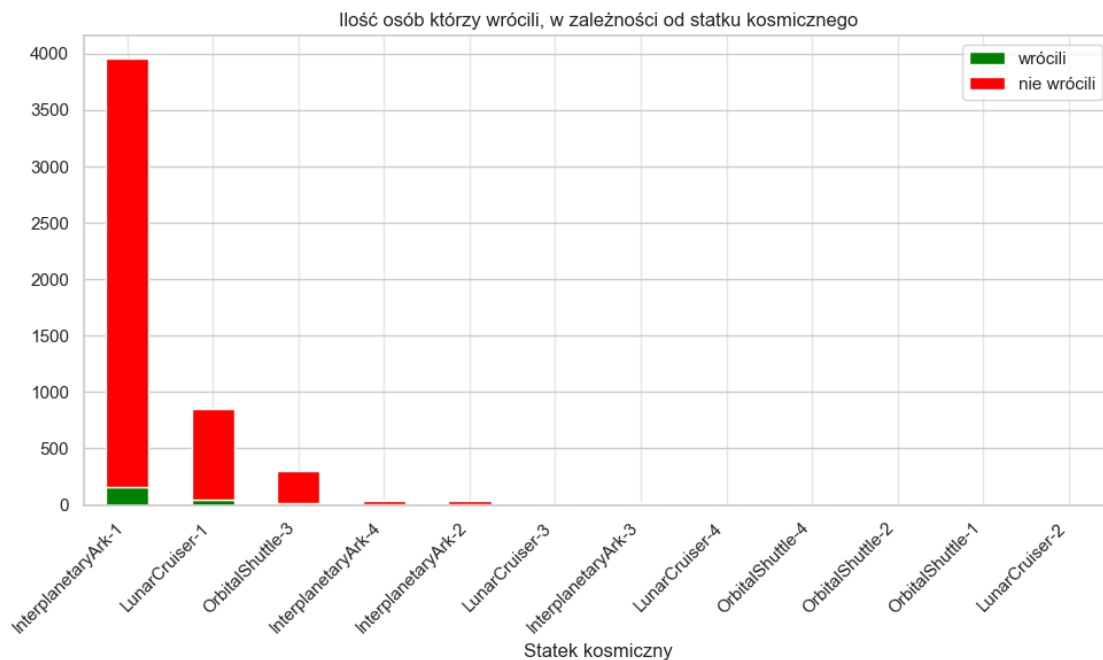
```

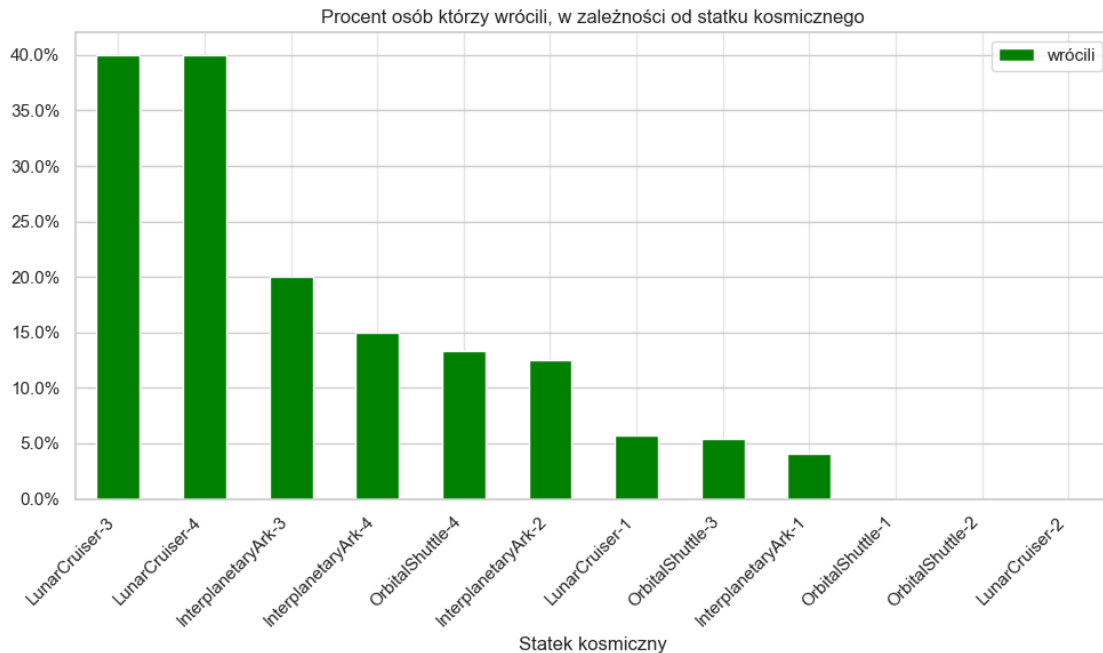
```

df_stats['returned_pct'] = df_stats['returned'] / df_stats['total']
df_stats['stopped_pct'] = df_stats['stopped'] / df_stats['total']
df_stats = df_stats.sort_values(by='returned_pct', ascending=False)

plot_df = df_stats[['returned_pct']]
ax = plot_df.plot(kind='bar', stacked=True, figsize=(10, 6), color=['green', 'red'])
xticks = range(len(plot_df))
labels = df_spaceships.loc[plot_df.index, 'name']
plt.xticks(ticks=xticks, labels=labels, rotation=45, ha='right')
ax.set_yticklabels(['{:, .1%}'.format(x) for x in ax.get_yticks()])
plt.title("Procent osób którzy wrócili, w zależności od statku kosmicznego")
plt.xlabel("Statek kosmiczny")
plt.legend(["wrócili", "nie wrócili"])
plt.tight_layout()
plt.grid(axis='x', alpha=0.5)
plt.show()

```





Zdecydowanie najpopularniejszym statkiem jest InterplanetaryArk-1 (3960 osób), a za nim są LunarCruiser-1 (850 osób) i OrbitalShuttle-3 (296 osób). Statki OrbitalShuttle-2 i Lunar-Cruiser-2 jeszcze wogóle nie wyruszyły z klientami, a z OrbitalShuttle-1 który wyleciał z 10 klientami, żaden nie zdecydował się na ponowną wycieczkę. Ze względu na małą ilość klientów zabranych statkami innymi niż 3 najpopularniejsze (do 40 osób) ciężko powiedzieć, czy rodzaj statku ma rzeczywisty wpływ na to czy klient wróci na kolejną wyprawę.

1.4 Dodatkowe pytania oraz ich analiza:

1.4.1 Jak wygląda rozkład zarobków w firmie w zależności od stanowiska.

```
[13]: employees_query = """SELECT
    role,
    salary,
    STDDEV(salary) AS salary_std
FROM
    Employees
GROUP BY
    role
ORDER BY
    role
"""
cursor.execute(employees_query)
employees = cursor.fetchall()
employees_df = pd.DataFrame(employees, columns=[i[0] for i in cursor.
    ↳description])
```

```

employees_df['salary'] = employees_df['salary'].astype(float)

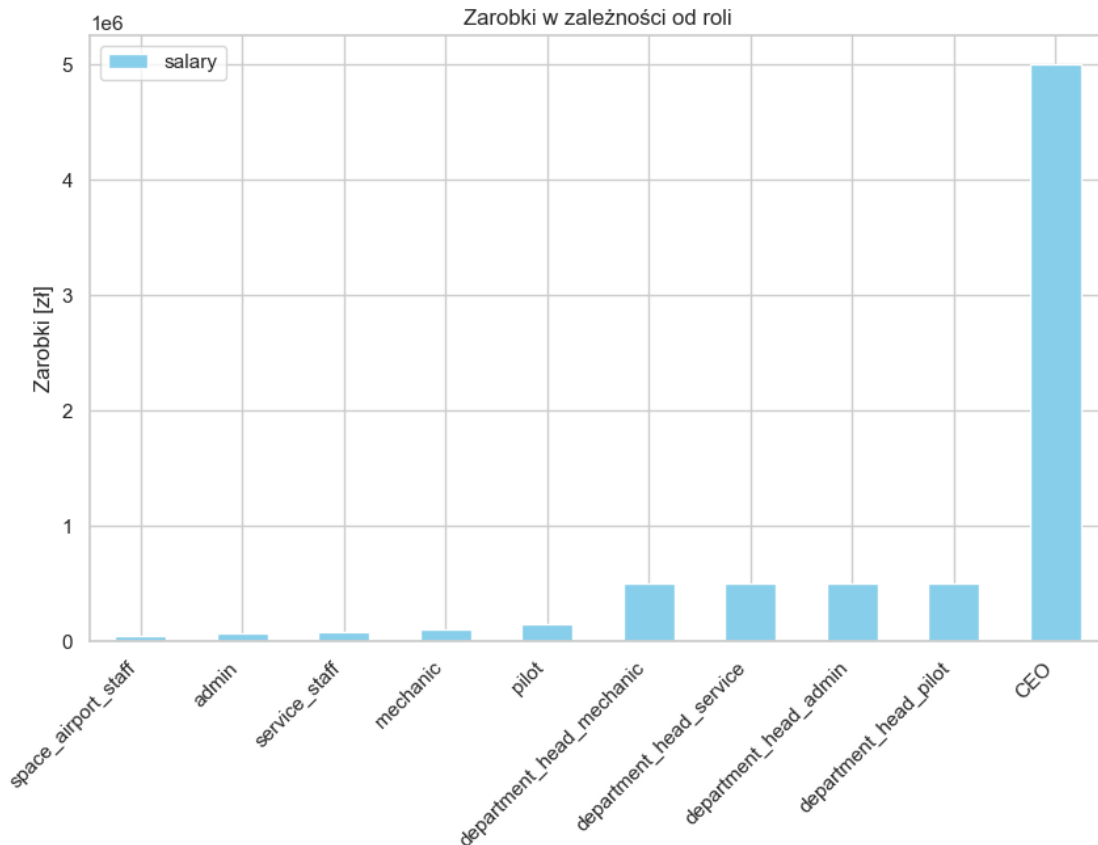
employees_df.sort_values(by='salary').plot(
    kind='bar',
    y='salary',
    x='role',
    title='Zarobki w zależności od roli',
    ylabel='Zarobki [zł]',
    figsize=(10, 6),
    xlabel='Rola',
    color='skyblue'
)

plt.xticks(rotation=45, ha='right');
plt.xlabel(None)

employees_salary_mean = employees_df['salary'].mean()
employees_salary_std = employees_df['salary'].std()

no_ceo_employees_salary_mean = employees_df[employees_df['role'] != 'CEO']['salary'].mean()
no_ceo_employees_salary_std = employees_df[employees_df['role'] != 'CEO']['salary'].std()

```



```
[ ]: display(Markdown(f"""#### Średnia i odchylenie standardowe zarobków
Średnie zarobki w firmie wynoszą {employees_salary_mean:.2f} zł, a
↳ odchylenie standardowe to {employees_salary_std:.2f} zł.
Natomiast bez uwzględnienia CEO średnia zarobków wynosi
↳ {no_ceo_employees_salary_mean:.2f} zł, a odchylenie standardowe to
↳ {no_ceo_employees_salary_std:.2f} zł.
Najmniej zarabiają pracownicy portu kosmicznego bo tylko
↳ {employees_df[employees_df['role'] == 'space_airport_staff']['salary'].
↳ values[0]:.2f} zł,
a najwięcej CEO, który zarabia {employees_df[employees_df['role'] ==
↳ 'CEO']['salary'].values[0]:.2f} zł. Jest to różnica dwóch rzędów wielkości.
↳ """))
```

Średnia i odchylenie standardowe zarobków Średnie zarobki w firmie wynoszą **745000.00 zł**, a odchylenie standardowe to **1509085.45 zł**. Natomiast bez uwzględnienia CEO średnia zarobków wynosi **272222.22 zł**, a odchylenie standardowe to **217760.06 zł**. Najmniej zarabiają pracownicy portu kosmicznego bo tylko **50000.00 zł**, a najwięcej CEO, który zarabia **5000000.00 zł**. Jest to różnica dwóch rzędów wielkości. Przyjrzyjmy się teraz wydajności pracowników. Który wypada najlepiej na tle pozostałych?

1.4.2 Wydajność pracowników - którzy wypadają najlepiej na tle pozostałych?

```
[15]: employees_performance_query = """
SELECT
    CONCAT(Employees.first_name, ' ', Employees.last_name) AS full_name,
    Employees.pesel,
    Employees.role,
    specialization,
    COUNT(RecoveryTasks.pesel) + COUNT(RepairTasks.pesel) + COUNT(TripCrew.
    ↪pesel) as ilosc_zadan
FROM
    Employees
LEFT JOIN
    RecoveryTasks ON Employees.pesel = RecoveryTasks.pesel
LEFT JOIN
    RepairTasks ON Employees.pesel = RepairTasks.pesel
LEFT JOIN
    TripCrew ON Employees.pesel = TripCrew.pesel
GROUP BY
    Employees.pesel
"""

cursor.execute(employees_performance_query)
employees_performance = cursor.fetchall()
employees_performance_df = pd.DataFrame(employees_performance, columns=[i[0] ↪
    ↪for i in cursor.description])
```

```
[16]: top_10_data = employees_performance_df.sort_values(by='ilosc_zadan', ↪
    ↪ascending=False).iloc[:10]

unique_roles = top_10_data['role'].unique()
colors = plt.get_cmap('tab10', len(unique_roles))
role_color_map = {role: colors(i) for i, role in enumerate(unique_roles)}

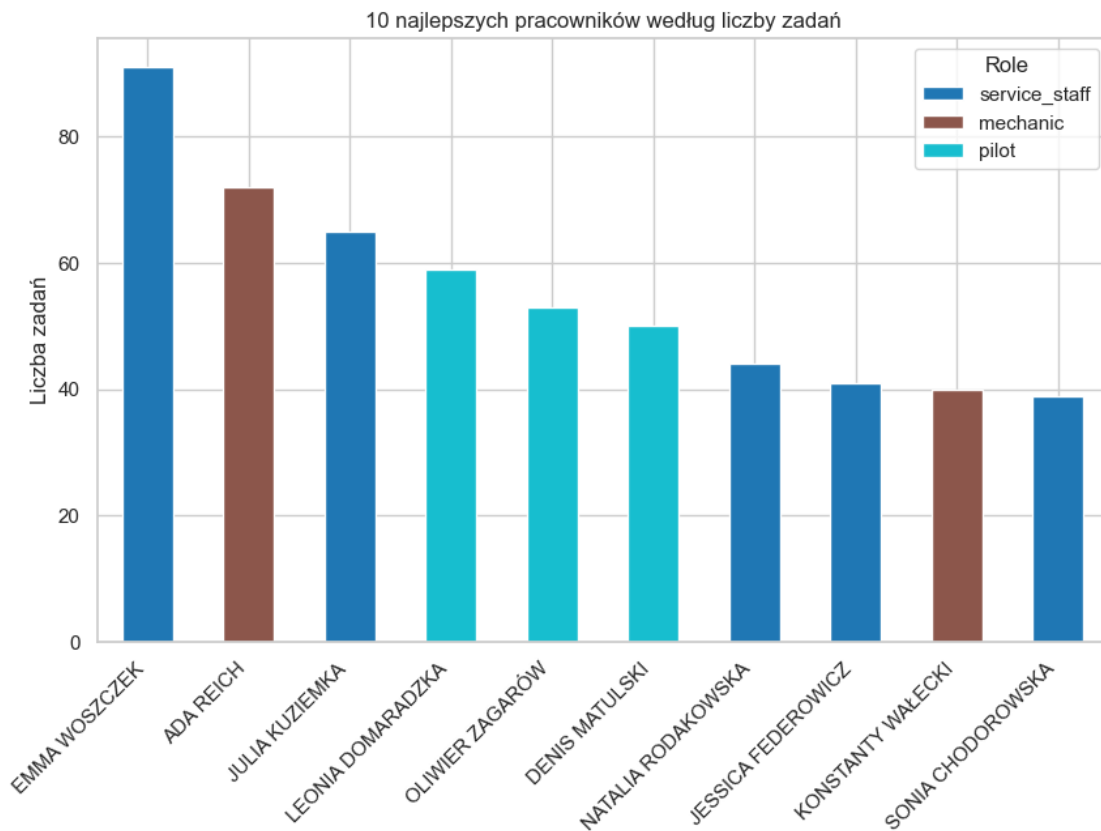
bar_colors = top_10_data['role'].map(role_color_map)

ax = top_10_data.plot(
    kind='bar',
    x='full_name',
    y='ilosc_zadan',
    title='10 najlepszych pracowników według liczby zadań',
    ylabel='Liczba zadań',
    xlabel='PESEL',
    figsize=(10, 6),
    color=bar_colors,
    legend=False
```

```
)

from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=role_color_map[role], label=role) for role_ in unique_roles]
ax.legend(handles=legend_elements, title='Role')

plt.xticks(rotation=45, ha='right');
plt.xlabel(None);
```



Zadania możemy podzielić na naprawę statku kosmicznego, odzyskiwanie rakiety oraz prowadzenie statku kosmicznego.

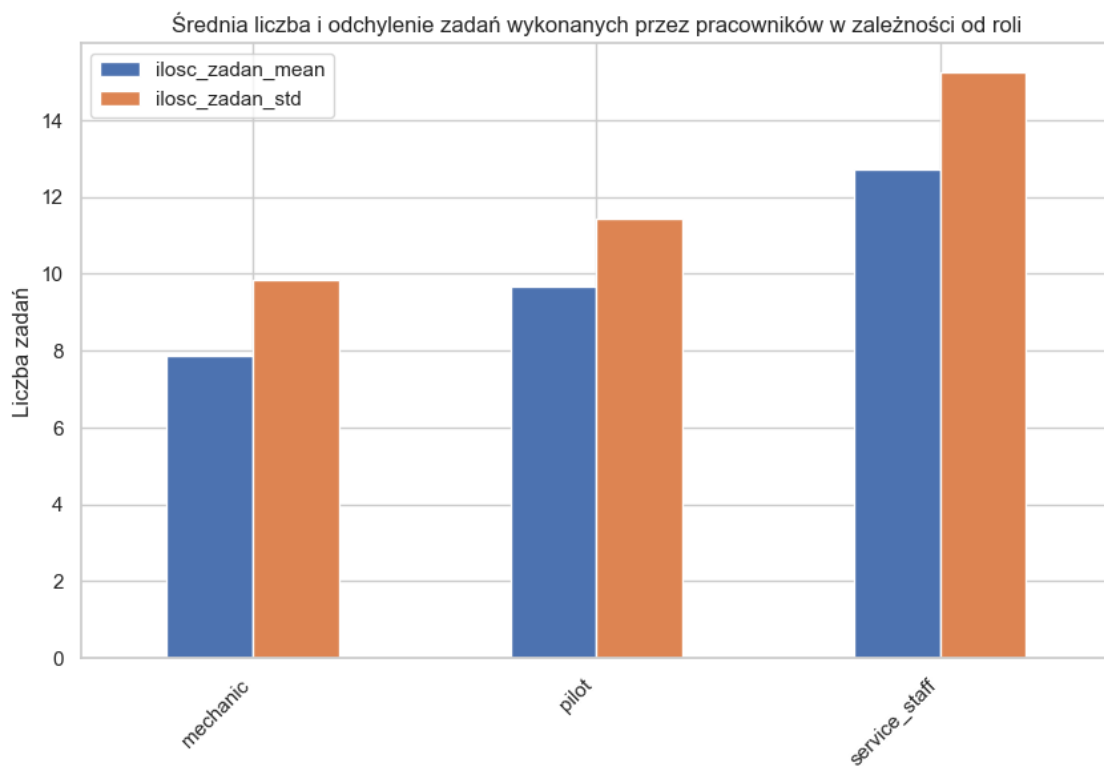
Z wykresu możemy wyczytać, że najwięcej zadań, bo ponad 80, w swojej karierze wykonała Emma Woszczek będąc przy okazji na stanowisku personelu obsługi. Zaraz po niej znajduje się Ada Reich na stanowisku mechanika. Natomiast pierwszym pilotem w rankingu jest Leonia Domaradzka. Spójrzmy teraz na te 3 grupy. Która z nich średnio wykonała najwięcej zadań?

```
[17]: employees_performance_df.where(
        lambda x: x['ilosc_zadan'] > 0).groupby('role').agg(
```

```

    ilosc_zadan_mean=('ilosc_zadan', 'mean'),
    ilosc_zadan_std=('ilosc_zadan', 'std')
).plot.bar(
    y=['ilosc_zadan_mean', 'ilosc_zadan_std'],
    title='Średnia liczba i odchylenie zadań wykonanych przez pracowników w
↳ zależności od roli',
    ylabel='Liczba zadań',
    xlabel='Rola',
    figsize=(10, 6),
)
plt.xticks(rotation=45, ha='right');
plt.xlabel(None);

```



Jak możemy zauważyć, średnio najwięcej zadań przymuje personel obsługi, mając jednocześnie największe odchylenie. Warto też wspomnieć, że zarobki personelu obsługi są najmniejsze z tej trójki.

1.4.3 Jaki statek był najdłużej wykorzystywany, a jaki ma największy stosunek wylotów do napraw?

```
[18]: ship_query = """
SELECT
    spaceship_id,
    name,
    type,
    manufacturing_date,
    status
FROM
    Spaceships
"""

repair_query = """
SELECT
    spaceship_id,
    COUNT(*) AS repair_count
FROM
    RepairTasks
GROUP BY
    spaceship_id
"""

cruise_query = """
SELECT
    spaceship_id,
    COUNT(*) AS cruise_count
FROM
    Trips
GROUP BY
    spaceship_id
"""

cursor.execute(ship_query)
ships = cursor.fetchall()
ships_df = pd.DataFrame(ships, columns=[i[0] for i in cursor.description])

ships_df['manufacturing_date'] = pd.to_datetime(ships_df['manufacturing_date'])

cursor.execute(repair_query)
repairs = cursor.fetchall()
repairs_df = pd.DataFrame(repairs, columns=[i[0] for i in cursor.description])

cursor.execute(cruise_query)
```

```

cruises = cursor.fetchall()
cruises_df = pd.DataFrame(cruises, columns=[i[0] for i in cursor.description])

ships_repairs_df = ships_df.merge(
    repairs_df,
    on='spaceship_id',
    how='left'
).fillna({'repair_count': 0})

ships_cruises_df = ships_repairs_df.merge(
    cruises_df,
    on='spaceship_id',
    how='left'
).fillna({'cruise_count': 0})

```

```

[19]: plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)

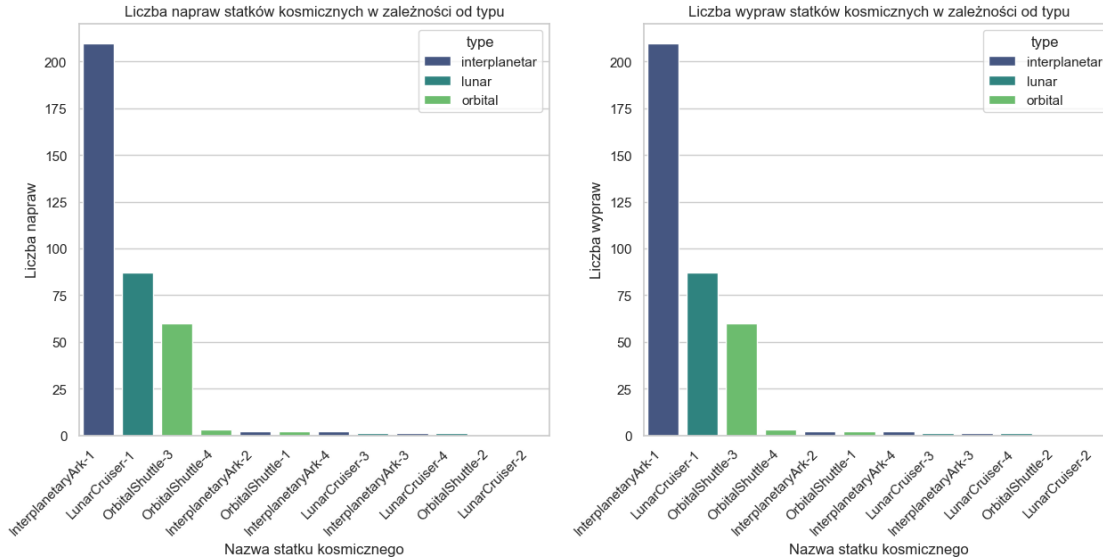
sns.barplot(
    data=ships_repairs_df.sort_values(by='repair_count', ascending=False),
    x='name',
    y='repair_count',
    hue='type',
    palette='viridis'
)

plt.title('Liczba napraw statków kosmicznych w zależności od typu')
plt.ylabel('Liczba napraw')
plt.xlabel('Nazwa statku kosmicznego')
plt.xticks(rotation=45, ha='right');

plt.subplot(1, 2, 2)
sns.barplot(
    data=ships_cruises_df.sort_values(by='cruise_count', ascending=False),
    x='name',
    y='cruise_count',
    hue='type',
    palette='viridis'
)

plt.title('Liczba wypraw statków kosmicznych w zależności od typu')
plt.ylabel('Liczba wypraw')
plt.xlabel('Nazwa statku kosmicznego')
plt.xticks(rotation=45, ha='right');

```

Najczęściej używanymi statkami ze swojej kategorii są *InterplanetaryArk-1*, *LunarCruiser-1* oraz *OrbitalShuttle-3*.

Wykresy ilości napraw statków i ilości wycieczek z ich użyciem są identyczne. Oznacza to, że po każdym zakończonym locie przeprowadzono naprawy statku. Nie ma więc statku który po przelocie byłby całkowicie sprawny, lub po prostu pojęcia naprawy i konserwacji statku jest dla firmy równoważne.

1.4.4 Ile wynosi średnie zapełnienie statków z zależności od celu podróży?

```
[3]: # 1. Połączenie SQLAlchemy
engine = create_engine(
    "mysql+mysqlconnector://team07:te%40mlot@giniewicz.it:3306/team07",
    pool_recycle=3600,
    pool_pre_ping=True
)

# =====
# A) baza: każdy lot + zapełnienie (%)
# =====

sql_base = text("""
SELECT
    t.trip_id,
    t.name AS trip_name,
    d.name AS destination,
    s.name AS spaceship_name,
    s.capacity,
    COUNT(tp.participant_id) AS passengers,
    ROUND(COUNT(tp.participant_id) / s.capacity * 100, 2) AS occupancy_pct

```

```

FROM Trips t
JOIN Spaceships s ON t.spaceship_id = s.spaceship_id
JOIN Destinations d ON t.destination_id = d.destination_id
LEFT JOIN TripParticipants tp ON t.trip_id = tp.trip_id
GROUP BY
    t.trip_id, t.name, d.name,
    s.name, s.capacity
""")

with engine.begin() as conn:
    df_occ = pd.read_sql(sql_base, conn)

# =====
# 1) Średnie zapełnienie wszystkich lotów
# =====
overall_avg = df_occ['occupancy_pct'].mean()
print(f"Średnie zapełnienie wszystkich statków: {overall_avg:.2f} %\n")

# =====
# 2) Ile lotów było pustych (0 pasażerów)
# =====
empty_flights = (df_occ['passengers'] == 0).sum()
print(f"Liczba lotów zupełnie pustych: {empty_flights}\n")

# =====
# 3) Destynacje vs średnie zapełnienie
# =====
df_dest = (df_occ
            .groupby('destination', as_index=False)['occupancy_pct']
            .mean()
            .sort_values('occupancy_pct'))

print("Średnie zapełnienie wg destynacji:")
display(df_dest, "\n")

# =====
# 4) Wykres - średnie zapełnienie wg destynacji
# =====
plt.figure(figsize=(9,6))
plt.barh(df_dest['destination'], df_dest['occupancy_pct'],
        color='cornflowerblue')
plt.xlabel("Średnie zapełnienie statku (%)")
plt.title("Porównanie średniego zapełnienia wg destynacji")
plt.xlim(0, 100)
for idx, val in enumerate(df_dest['occupancy_pct']):
    plt.text(val + 1, idx, f"{val:.1f} %", va='center')
plt.tight_layout()

```

```
plt.show()
```

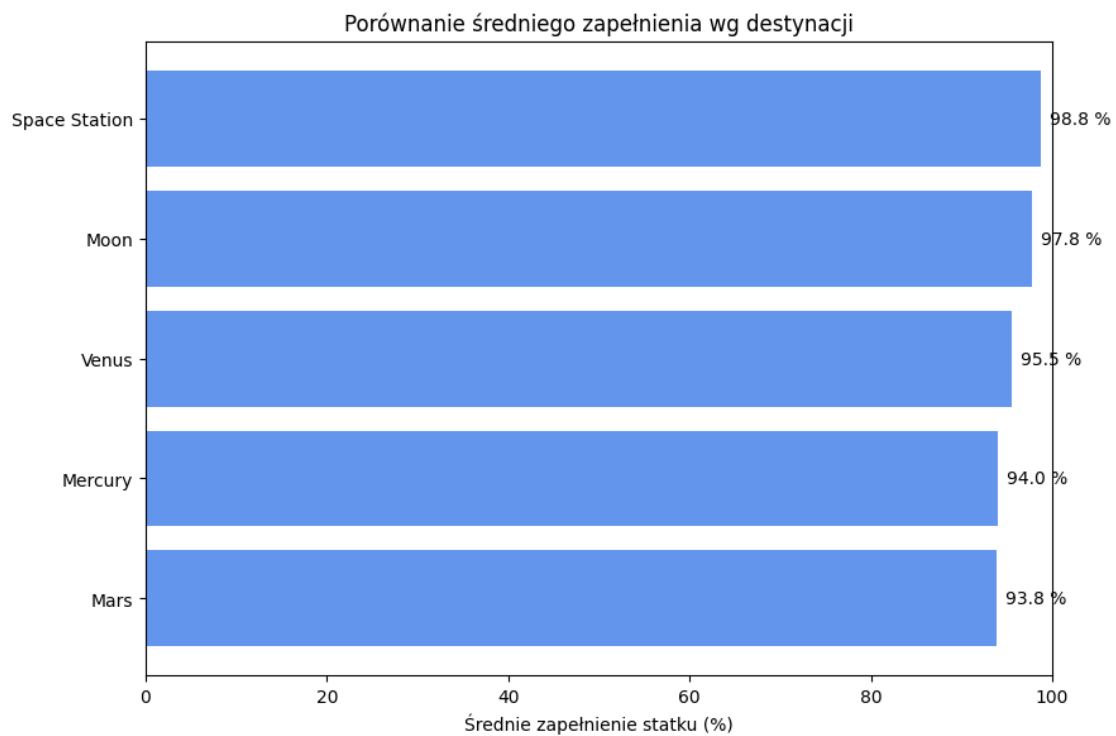
Średnie zapełnienie wszystkich statków: 95.99 %

Liczba lotów zupełnie pustych: 0

Średnie zapełnienie wg destynacji:

	destination	occupancy_pct
0	Mars	93.835616
1	Mercury	94.000000
4	Venus	95.522388
2	Moon	97.752809
3	Space Station	98.769231

'\n'



Średnie zapełnienie statków jest wysokie dla każdego z dostępnych celów podróży - ponad 90 %. Największe jest dla stacji kosmicznej i wynosi 98.8 % a najmniejsze dla Marsa równe 93.8 %.

1.4.5 Jaki wpływ członkowie załogi mają na poziom satysfakcji klientów podczas lotów?

```
[9]: sql = text("""
SELECT *
FROM (
    SELECT
        e.pesel                AS employee_id,
        CONCAT(e.first_name,' ',e.last_name) AS employee_name,
        tc.role,
        COUNT(DISTINCT tc.trip_id)          AS trips_handled,
        ROUND(AVG(tp.satisfaction_level),2)  AS avg_satisfaction
    FROM TripCrew tc
    JOIN Employees e ON e.pesel = tc.pesel      -- ← poprawione
    JOIN TripParticipants tp ON tp.trip_id = tc.trip_id
    GROUP BY
        e.pesel, e.first_name, e.last_name, tc.role
) AS t
WHERE trips_handled >= 10
ORDER BY avg_satisfaction DESC, trips_handled DESC
""")

with engine.begin() as conn:
    df_crew = pd.read_sql(sql, conn)

if df_crew.empty:
    print("Brak danych lub nikt nie obsłużył >= 10 wypraw.")
else:
    display(df_crew)

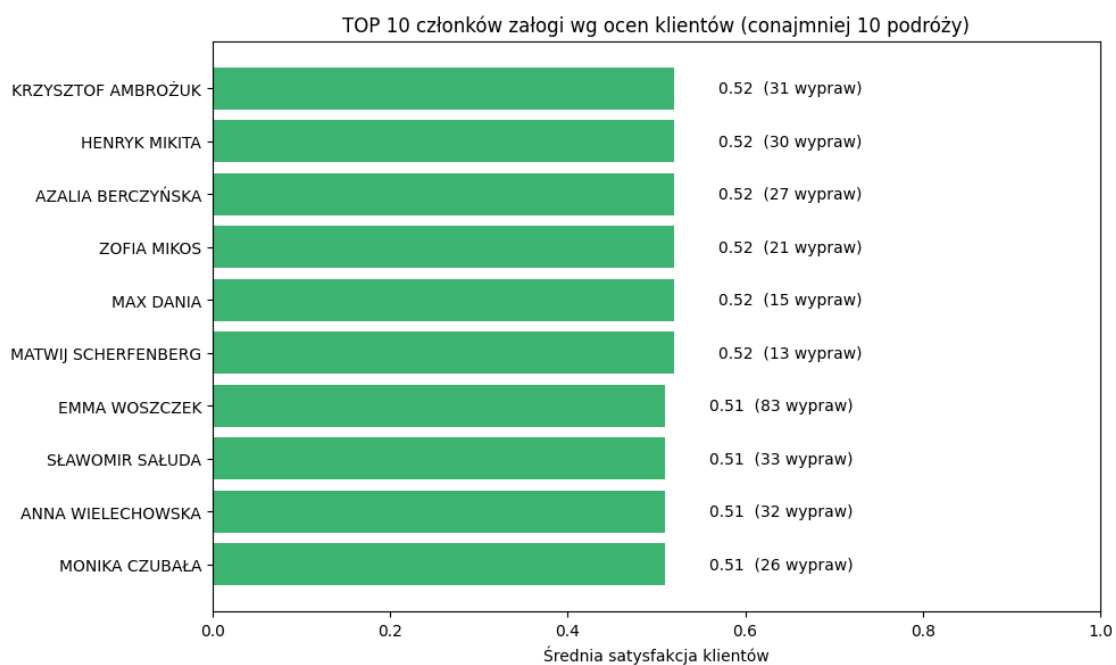
    top = df_crew.head(10)
    plt.figure(figsize=(10,6))
    bars = plt.barh(top['employee_name'], top['avg_satisfaction'],
                    color='mediumseagreen')
    plt.gca().invert_yaxis()
    plt.xlabel("Średnia satysfakcja klientów")
    plt.title("TOP 10 członków załogi wg ocen klientów (conajmniej 10 podróży)")
    for bar, trips in zip(bars, top['trips_handled']):
        plt.text(bar.get_width()+0.05,
                 bar.get_y()+bar.get_height()/2,
                 f"{bar.get_width():.2f}  ({trips} wypraw)",
                 va="center")
    plt.xlim(0, 1)
    plt.tight_layout()
    plt.show()
```

	employee_id	employee_name	role	trips_handled \
0	20430747551	KRZYSZTOF AMBROŻUK	service_staff	31

1	21452786993	HENRYK MIKITA	service_staff	30
2	23412848440	AZALIA BERCZYŃSKA	service_staff	27
3	21502577186	ZOFIA MIKOS	pilot	21
4	44470921058	MAX DANIA	service_staff	15
..	...	...	...	...
63	41410318183	JADWIGA WALCZUKIEWICZ	pilot	28
64	19473057017	NATANAEL GODNARSKI	service_staff	22
65	23502423696	JÓZEF NIEZNAJ	pilot	11
66	43471515138	DAVID SZEWCZYKOWSKI	pilot	11
67	17442003449	DANUTA LISICA	pilot	10

avg_satisfaction	
0	0.52
1	0.52
2	0.52
3	0.52
4	0.52
..	...
63	0.46
64	0.46
65	0.46
66	0.46
67	0.46

[68 rows x 5 columns]



Poziom satysfakcji klientów podczas lotu, dla każdego członka załogi, który odbył minimum 10 wypraw, znajduje się w przedziale jest pomiędzy 45% a 55%. Przedział ten jest bardzo mały i trudno stwierdzić czy rzeczywiście istnieje zależność między członkiem załogi, poziomem satysfakcji klienta z lotu.